

# ¿Qué es la teoría de la computación?

Buscando en internet encontré estos autores que cada uno tiene su propia definición desde su propio enfoque:

**César Jesús Pardo Calvache, Siler Amador Donado y Katerine Márceles Villalba (2023):** «La teoría de la computación o teoría de la informática se centra en el estudio y formalización de procesos abstractos, esto con el objetivo de facilitar su reproducción a través del uso de sistemas más claros y explícitos, por ejemplo: con el uso de símbolos y reglas lógicas. [...] Permite establecer las capacidades, pero también limitaciones de las computadoras, por ejemplo: comprender los procesos de cálculo de una computadora en función de la dificultad de cómputo, y de esta manera, establecer las limitaciones de ciertos procesos y su complejidad».

**Gabriel Gerónimo Castillo:** «Es la rama de las matemáticas y la informática teórica que estudia los límites de lo que se puede computar, utilizando modelos abstractos de computación como las máquinas de Turing y los autómatas finitos». Asimismo, señala que está formada principalmente por tres subtemas: autómatas/lenguajes, computabilidad y complejidad computacional.

Pero basándonos en estas definiciones, podremos construir una propia que se asemeje a lo que cada autor nos quiere decir:

“La Teoría de la Computación es la disciplina fundamental de la ciencia de la informática y las matemáticas teóricas encargada de estudiar, formalizar y abstraer los procesos de cálculo mediante modelos matemáticos y reglas lógicas, determinando de este modo las capacidades y los límites de los sistemas de cómputo.”

Y cada autor se plantea diferentes problemas, pero de manera general las preguntas que se plantean son las siguientes:

- ¿Qué problemas pueden ser resueltos mediante un algoritmo?
- ¿Qué puede hacer una computadora?
- ¿Qué puede hacer una computadora eficientemente?

# Programa de Hilbert

Antes de entrar en el tema principal, vamos a hablar un poco sobre quien era David Hilbert.

Fue un matemático alemán muy importante e influyente en los siglos XIX y XX, nació el 23 de enero de 1862, estableció su reputación como gran matemático y científico, inventando y desarrollando ideas que revolucionaron el universo matemático de la época

Hilbert trabajó como profesor en la Universidad de Königsberg de 1886 a 1895, posteriormente, obtuvo el puesto de Catedrático de Matemática en la Universidad de Göttingen, que en aquella fecha era el mejor centro de investigación matemática en el mundo; aquí permanecería el resto de su vida.

Durante su estancia en la universidad de Göttingen, David Hilbert enfocaba su atención en resolver las grandes problemáticas en las matemáticas de ese entonces, tales como:

- **Las paradojas en la Teoría de Conjuntos**
- **La Controversia del Infinito**
- **El Intuicionismo**

Al encontrarse estas problemáticas, Hilbert, ya consagrado, se propuso a “salvar” la matemática, a esta propuesta de salvar la matemática se le llama “Programa de Formalización”. El programa de Hilbert no fue un esfuerzo en solitario, sino un proyecto que atrajo a grandes mentes, juntos formaron lo que a menudo se llama la “Escuela de Hilbert”.

Su objetivo principal era asegurar la absoluta certeza y rigor de las matemáticas mediante su formalización total en un sistema axiomático finito. El programa exigía demostrar formalmente tres metas clave:

- **La Completitud:** Probar que cualquier proposición matemática verdadera podía demostrarse dentro del sistema.
- **La Consistencia (o Coherencia):** Demostrar que el sistema era libre de contradicciones.
- **La Decidibilidad (*Entscheidungsproblem*):** Encontrar un procedimiento mecánico o algoritmo general que permitiera determinar la veracidad o falsedad de cualquier sentencia matemática.

Después entre 1935 y 1937, de forma casi simultánea e independiente, Alonzo Church en los EE. UU. y Alan Turing en el Reino Unido, publican sus trabajos donde demuestran que la tercera pregunta de Hilbert, el Entscheidungsproblem o problema de la decisión, no tiene solución.

Ambas demostraciones comprobaron de forma definitiva que no existe un algoritmo general para la decisión en los sistemas formales. Este hallazgo revolucionó la lógica y dio origen formal a la computación moderna, al demostrar que los procesos computacionales y sus límites intrínsecos podían describirse rigurosamente a través de máquinas abstractas y símbolos matemáticos.

La teoría de la computación se divide fundamentalmente en tres grandes áreas o ramas que permiten estructurar el análisis formal de los sistemas de cómputo:

## **Computabilidad**

Es la parte de la computación que estudia los problemas de decisión que se pueden resolver con un algoritmo o equivalentemente con una máquina de Turing, se originó en la década de los años 30 con los trabajos de los lógicos Church, Gödel, Kleene, Post y Turing.

La teoría de la computabilidad estudia qué lenguajes son decidibles con diferentes tipos de máquinas, también pretende abstraer los detalles de los sistemas computacionales y busca un algoritmo para efectuar un cálculo sin preocuparse por los detalles de implantación; en este caso se dice que la función es computable o calculable. Puede verificarse si una función es computable utilizando una máquina de Turing como un modelo de máquina isomorfa a cualquier otro sistema computacional.

¿Cuántos recursos de tiempo y espacio necesita una computadora para resolver un problema de forma eficiente (problemas tratables frente a intratables)?

## **Complejidad**

La teoría formaliza esta intuición al introducir modelos matemáticos de computación para estudiar estos problemas y cuantificar su complejidad computacional, es decir, se centra en clasificar los problemas computacionales según el uso de recursos y relacionar estas clases entre sí, También uno de los roles de la teoría de la complejidad computacional es determinar los límites prácticos de lo que las computadoras pueden y no pueden hacer.

Y la pregunta que responde es:

¿Qué problemas puede resolver una máquina de Turing y cuáles son intrínsecamente incomputables?

# Lenguajes Formales

Un lenguaje formal es un conjunto de cadenas de caracteres que siguen una serie de reglas sintácticas. Estas reglas se definen mediante una gramática formal, que es un conjunto de reglas que indican cómo se deben combinar los elementos del lenguaje para formar cadenas válidas. Los lenguajes formales y los autómatas están estrechamente relacionados. De hecho, se puede demostrar que cualquier lenguaje formal puede ser reconocido por un autómata y viceversa.

Los lenguajes formales se pueden clasificar en cuatro tipos principales, cada uno con sus propias características y capacidades.

- El primer tipo, los lenguajes libres o recursivamente enumerables (Tipo 0), utilizan gramáticas libres y permiten la recursión, lo que significa que una regla de producción puede referirse a sí misma.
- El segundo tipo, los lenguajes independientes del contexto (Tipo 2), son reconocidos por gramáticas libres de contexto, por lo que permiten el reemplazo de símbolos no terminales sin considerar el contexto circundante, lo que los hace adecuados para la descripción de estructuras que no dependen del entorno.
- El tercer tipo, los lenguajes dependientes del contexto (Tipo 1), requieren un contexto específico para el reemplazo de símbolos no terminales. Esto significa que la misma regla de producción puede tener diferentes significados dependiendo del símbolo que la rodea.
- Y por último lenguajes regulares o lineales (Tipo 3), generados por gramáticas regulares, se caracterizan por dependencias lineales en las cadenas. Estos lenguajes son los más simples de analizar y diseñar, y se utilizan para describir patrones simples, como direcciones de correo electrónico o números de teléfono.

Y su pregunta que responde es

¿Qué tipo de estructura y reglas gramaticales posee un lenguaje y qué modelo mecánico mínimo se requiere para reconocerlo?

# Tesis de Church-Turing

Afirmación sostenida intuitivamente, y no probada formalmente, por Alonzo Church y Alan Turing, hacia 1937, según la cual existe un algoritmo para la solución de un problema matemático si y sólo si existe una máquina de Turing que pueda computar dicho problema. Church formuló esta tesis mediante el llamado «cálculo de lambda». La tesis lleva implícita la afirmación de que la mente humana es una máquina de Turing, o lo que es lo mismo, de que el pensamiento humano es computable, o que la mente es un modelo computacional.

Aunque se asume como cierta, la tesis de Church-Turing no puede ser probada ya que no se poseen de los medios necesarios, por eso es una tesis. Ello debido a que “procedimiento efectivo” y “algoritmo” no son conceptos dentro de ninguna teoría matemática y no son definibles fácilmente.

Pero sostiene que este conjunto comprende la totalidad de aquellas funciones cuyos valores pueden obtenerse mediante un método efectivo que cumpla con condiciones de eficacia. Como consecuencia, se establece que si una Máquina de Turing no es capaz de resolver un problema, ningún otro computador podrá hacerlo, lo que demuestra que dichas limitaciones corresponden intrínsecamente a la naturaleza de los procesos computacionales y no a restricciones tecnológicas.

## Referencias:

**Libro base de la materia (Teoría de lenguajes formales):** Balari, S. (2014). Teoría de lenguajes formales: Una introducción para lingüistas. Universitat Autònoma de Barcelona; Centre de Lingüística Teòrica.

**Libro de enfoque práctico (Teoría de la computación):** Pardo Calvache, C. J., Amador Donado, S., & Márceles Villalba, K. (2023). *Teoría de la computación: Lenguajes y autómatas, un enfoque práctico*. Sello Editorial Uniautónoma del Cauca; Institución Universitaria Colegio Mayor del Cauca.

<https://doi.org/10.37554/9789588614755>

**Material de estudio y resumen sobre Computabilidad:** De Marco, F., Soto, L., & Martínez, D. (s. f.). *Computabilidad: Fundamentos teóricos de la informática*. [Documento PDF].

**Artículo web sobre el Programa de Hilbert:** Romero, L. (2025, 8 de octubre). ¿En qué consistió el Programa de Hilbert y cómo influyó en la teoría de la demostración? *Revista Columnas*. <https://revistacolumnas.mx/2025/10/08/en-que-consistio-el-programa-de-hilbert-y-como-influyo-en-la-teoria-de-la-demostracion/>

**Artículo web sobre el Entscheidungsproblem:** Colegio de Matemáticas Bourbaki. (2025, 2 de abril). *El Entscheidungsproblem y el inicio de la computación*. Blog del Colegio Bourbaki. <https://www.colegio-bourbaki.com/blog/entscheidungsproblem-y-el-inicio-de-la-computacion>

# Alfabeto, Cadena, Lenguaje y sus Operaciones Formales

**Alfabeto:** Un alfabeto es un conjunto finito de símbolos, por lo tanto, es un conjunto no vacío. Se denota comúnmente con la letra griega  $\Sigma$  (Sigma) o  $\Gamma$  (Gamma). Por ejemplo,  $\Sigma = \{a, b, c\}$ .

**Cadena:** Una cadena es una sucesión lineal de elementos enlazados entre sí a partir de un alfabeto.

**Lenguaje ( $L$ ):** Es un conjunto de cadenas formadas con los símbolos de un alfabeto determinado. Los autómatas finitos, por ejemplo, son capaces de reconocer únicamente los llamados *Lenguajes Regulares*.

## Operaciones sobre Cadenas y sobre Lenguajes

- **Concatenación de cadenas:** Consiste en unir dos cadenas consecutivamente (por ejemplo, dada la cadena  $x$  y la cadena  $y$ , su concatenación se escribe  $xy$ ). Posee la propiedad asociativa.
- **Potencia de un alfabeto o cadena:** Consiste en multiplicar o concatenar un alfabeto o cadena consigo mismo un número  $n$  de veces (denotado como  $\Sigma^n$ ).
- **Reflexión (o palabra inversa):** Operación que invierte el orden de los símbolos que componen una cadena.
- **Unión entre lenguajes:** Operación conjuntista que agrupa todas las cadenas pertenecientes a dos o más lenguajes.
- **Intersección:** Operación que resulta en el conjunto de cadenas que son comunes a dos lenguajes dados.
- **Diferencia:** Operación que obtiene las cadenas que pertenecen a un lenguaje, pero no al otro.
- **Cerradura de Kleene ( $\Sigma^*$ ):** Conjunto de todas las cadenas posibles de cualquier longitud (incluyendo longitud cero) que se pueden formar con los símbolos de un alfabeto  $\Sigma$ .
- **Cerradura positiva ( $\Sigma^+$ ):** Conjunto de todas las cadenas posibles formadas con los símbolos de un alfabeto  $\Sigma$ , **excluyendo** la cadena vacía ( $\lambda$  o  $\epsilon$ ).

## Explicación de $\Sigma^0 = \{\lambda\}$

- Por definición matemática en la teoría de la computación, cualquier alfabeto o cadena elevado a la potencia cero ( $\Sigma^0$ ) da como resultado un conjunto que contiene únicamente a la **cadena vacía** ( $\lambda$  o  $\varepsilon$ ). Esto representa el elemento neutro de la concatenación, es decir, la cadena de longitud cero.

## ¿Qué distingue a $\Sigma^*$ de $\Sigma^+$ ?

- **Clausura de Kleene ( $\Sigma^*$ ):** Incluye todas las cadenas posibles formadas con los símbolos del alfabeto, **incluyendo la cadena vacía** ( $\lambda$  o  $\varepsilon$ ). Su definición matemática es  $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$
- **Clausura positiva ( $\Sigma^+$ ):** Incluye exactamente las mismas combinaciones de cadenas, con la única y estricta diferencia de que **excluye la cadena vacía** ( $\lambda$  o  $\varepsilon$ ). Su definición formal es  $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$  (lo que equivale a decir que  $\Sigma^+ = \Sigma^* - \{\lambda\}$ ).

## La jerarquía de Chomsky

| <b>Tipo</b> | <b>Tipo de Lenguaje</b>                                      | <b>Gramática</b>                                              | <b>Máquina que lo reconoce</b>                          |
|-------------|--------------------------------------------------------------|---------------------------------------------------------------|---------------------------------------------------------|
| Tipo 3      | Lenguajes Regulares                                          | Gramáticas Regulares<br>(o lineales por la derecha/izquierda) | Autómata Finito<br>(Determinístico y No Determinístico) |
| Tipo 2      | Lenguajes Independientes del Contexto (o Libres de Contexto) | Gramáticas Independientes del Contexto                        | Autómata de Pila                                        |
| Tipo 1      | Lenguajes Sensibles al Contexto                              | Gramáticas Sensibles al Contexto                              | Autómata Linealmente Acotado                            |
| Tipo 0      | Lenguajes Recursivamente Enumerables                         | Gramáticas Sin Restricciones (o de Frase General)             | Máquina de Turing                                       |

# Autómatas Finitos y Expresiones Regulares

Un Autómata Finito Determinístico es un modelo matemático formal que representa un sistema con un número finito de estados. Se define formalmente como una 5-tupla:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Donde:

- $Q$ : Un conjunto finito y no vacío de estados.
- $\Sigma$ : Un alfabeto finito de símbolos de entrada.
- $\delta$ : Una función de transición que mapea  $Q \times \Sigma \rightarrow Q$ . Esto significa que, dado un estado actual y un símbolo de entrada, el autómata pasa a **un único y predecible** estado siguiente.
- $q_0$ : El estado inicial, donde  $q_0 \in Q$ .
- $F$ : Un conjunto de estados de aceptación o finales, donde  $F \subseteq Q$ .

## Definición formal del AFN (Autómata Finito No Determinístico)

Un Autómata Finito No Determinístico es similar a un AFD, pero con una flexibilidad mayor: a partir de un estado y un símbolo de entrada, el autómata puede transicionar a cero, uno o varios estados, o incluso realizar transiciones espontáneas sin consumir símbolos de entrada (usando la cadena vacía  $\lambda$  o  $\varepsilon$ ). Se define formalmente como una 5-tupla:

$$M = (Q, \Sigma, \delta, q_0, F)$$